PROJECT REPORT

CarbonX — Blockchain-Based Carbon Credit Trading Platform

1. Abstract

CarbonX is a decentralized blockchain application for managing the lifecycle of simulated digital carbon credits. It supports authorized issuance, on-chain verification, marketplace listing, purchase, ownership transfer, listing cancellation, and permanent retirement. Smart-contract rules enforce administrator, issuer, and owner permissions and prevent retired credits from being transferred or listed. The implementation uses Solidity and Hardhat for blockchain development and React, Vite, Ethers.js, and MetaMask for Web3 interaction. This is an educational prototype and does not represent legally verified or certified carbon offsets.

2. Introduction

Carbon markets require reliable records of **issuance, ownership, transfer, trading, and retirement**. Managing these activities across different systems can make it difficult to maintain a transparent and traceable history of a carbon credit. CarbonX demonstrates how blockchain technology can provide a **transparent, tamper-resistant, and traceable digital record** for these activities.

CarbonX represents each carbon credit through a **Solidity smart contract**, storing important information such as Credit ID, project name, project type, location, vintage year, tonnes of CO_2e, issuer, owner, metadata, and credit status. The platform supports authorized issuance, credit verification, marketplace listing, purchasing, listing cancellation, ownership transfer, and permanent retirement.

The application combines **React, Vite, Ethers.js, MetaMask, Solidity, and Hardhat** to create a complete Web3 workflow. Smart-contract rules control permissions and lifecycle transitions, while retired credits are prevented from being transferred or listed again. CarbonX is developed as an **educational blockchain prototype using simulated carbon-credit data**, demonstrating the technical possibilities of blockchain-based carbon-credit management.

3. Problem Statement

Traditional digital workflows for environmental assets often involve **multiple parties, databases, and records** to establish ownership, verify transactions, and maintain the history of an asset. This can make it difficult to maintain a consistent and transparent record throughout the complete lifecycle of a carbon credit.

CarbonX addresses this challenge by providing a **shared blockchain-based source of truth** where important credit information and state changes are recorded through a Solidity smart contract. Ownership, marketplace transactions, transfers, and retirement actions are validated by predefined smart-contract rules, providing a transparent and traceable digital lifecycle for each carbon credit.

4. Objectives

- Create a blockchain-based model for digital carbon-credit lifecycle management.
- Restrict credit issuance to authorized issuers.
- Track credit ownership on-chain.
- Provide a marketplace for listing and purchasing active credits.
- Allow owners to cancel listings and transfer eligible credits.
- Provide permanent retirement with a recorded retirement reason.
- Prevent retired credits from being transferred or listed.
- Demonstrate Web3 wallet interaction through MetaMask.

5. Proposed Solution

CarbonX combines a **Solidity smart contract** with a **React-based frontend** to provide a user-friendly interface for interacting with blockchain-based carbon credits. The smart contract manages core operations such as credit issuance, ownership, marketplace listings, purchases, transfers, cancellations, and retirement.

Ethers.js acts as the communication layer between the React frontend and the deployed smart contract, while **MetaMask** provides wallet connectivity and allows users to approve state-changing blockchain transactions. **Hardhat** provides the local blockchain development and testing environment, enabling the smart contract and frontend to be developed and tested securely before deployment to a public network.

6. System Architecture

React + Vite → Ethers.js → MetaMask → RPC → CarbonCreditTrading.sol → Hardhat Local Network

- **React + Vite:** Provides the CarbonX interface where users perform credit-related operations.
- **Ethers.js:** Acts as the bridge between the frontend and the blockchain smart contract.
- **MetaMask:** Connects user wallets and confirms blockchain transactions.
- **RPC:** Provides the communication path between the application and the local blockchain.
- **CarbonCreditTrading.sol:** Contains the main business logic for issuing, listing, buying, transferring, cancelling, and retiring carbon credits.
- **Hardhat Local Network:** Runs the local Ethereum-compatible blockchain where the smart contract is deployed and tested.

7. Actors and Roles

- Administrator — manages authorized issuers.
- Authorized Issuer — issues new carbon credits and assigns an initial owner.
- Credit Owner — can list, cancel listings, transfer, and retire eligible credits.
- Buyer — can inspect active listings and purchase a listed credit.

8. Carbon Credit Data Model

Field	Purpose
Credit ID	Unique identifier
Project Name	Carbon project name
Project Type	Project category
Location	Project location
Vintage Year	Credit vintage year
Tonnes CO2e	Quantity represented by the credit
Issuer	Authorized issuer wallet
Owner	Current owner wallet
Metadata Hash	Metadata/IPFS reference
Status	Current lifecycle state

9. Credit Issuance

- Only an authorized issuer can issue a credit.
- Project name is required.
- Tonnes CO2e must be greater than zero.
- Owner address must be valid.
- Vintage year must be within the contract's accepted range.
- The credit starts in ACTIVE status.
- A CreditIssued event is emitted.

10. Marketplace Logic

- Only the current owner can list a credit.
- A listing requires a price greater than zero.
- Retired credits cannot be listed.
- An active listing records the credit ID, seller, price, and active state.
- Only the listing seller can cancel the listing.
- A buyer must pay exactly the listed price.
- After a successful purchase, ownership changes to the buyer and the listing becomes inactive.

11. Transfer Logic

- Only the current owner can transfer a credit.
- The destination address must be valid.
- Retired credits cannot be transferred.
- Listed credits must first have their listing cancelled.
- Ownership is updated on-chain and a transfer event is emitted.

12. Retirement Logic

- Only the current owner can retire a credit.

- A listed credit must first have its listing cancelled.
- A retirement reason is required.
- The status changes to RETIRED.
- A retirement timestamp is recorded.
- Active supply is reduced and retired supply is increased.
- A CreditRetired event is emitted.
- Retired credits cannot be transferred or listed.

13. Security and Access Control

- Administrator-only issuer management.
- Authorized-issuer-only issuance.
- Owner-only credit operations.
- Listing validation before purchase or cancellation.
- Retired-credit protection.
- Non-reentrancy protection on the purchase function.

14. Technology Stack

- Frontend: React, Vite, JavaScript, HTML, CSS
- Blockchain: Solidity, Hardhat, Ethereum-compatible local network
- Web3: Ethers.js, MetaMask
- Development: Node.js, npm, TypeScript, Git, GitHub, Visual Studio Code

15. Implementation and Execution

1. **Install Root and Frontend Dependencies**
   Install all required blockchain dependencies in the project root and frontend dependencies inside the frontend directory using npm install.
2. **Compile the Smart Contract**
   Compile CarbonCreditTrading.sol using Hardhat to check for Solidity errors and generate the required contract artifacts.
   ➔ npx hardhat compile
3. **Start the Hardhat Local Blockchain**
   Start the local Ethereum-compatible blockchain to provide test accounts, transactions, and a development environment.
   ➔ npx hardhat node
4. **Deploy the Smart Contract**
   Deploy CarbonCreditTrading.sol to the running Hardhat network using the deployment script.
   ➔ npx hardhat run scripts/deploy.ts --network localhost
5. **Configure MetaMask**
   Connect MetaMask to the Hardhat local network using:
   - RPC URL: http://127.0.0.1:8545

- Chain ID: 31337

Import a Hardhat test account into MetaMask for performing transactions.

6. **Connect the React Frontend**
   The React application uses **Ethers.js** to connect to MetaMask and interact with the deployed smart contract. The deployed contract address and ABI are used by the frontend.
7. **Execute Admin and Issuer Workflows**
   The administrator can manage authorized issuers. Authorized issuers can then issue new carbon credits by providing project details, vintage year, tonnes of CO_2e, owner address, and metadata.
8. **Issue and Verify Credits**
   After issuance, each credit receives a unique Credit ID. Users can enter the Credit ID to retrieve and verify its project details, issuer, owner, and current status from the blockchain.
9. **Test Complete Credit Operations**
   The application is tested through the complete lifecycle, including **listing credits, checking listings, purchasing credits, cancelling listings, transferring ownership, and retiring credits**. Additional validation confirms that retired credits cannot be transferred or listed again.

16. Testing and Demonstrated Results

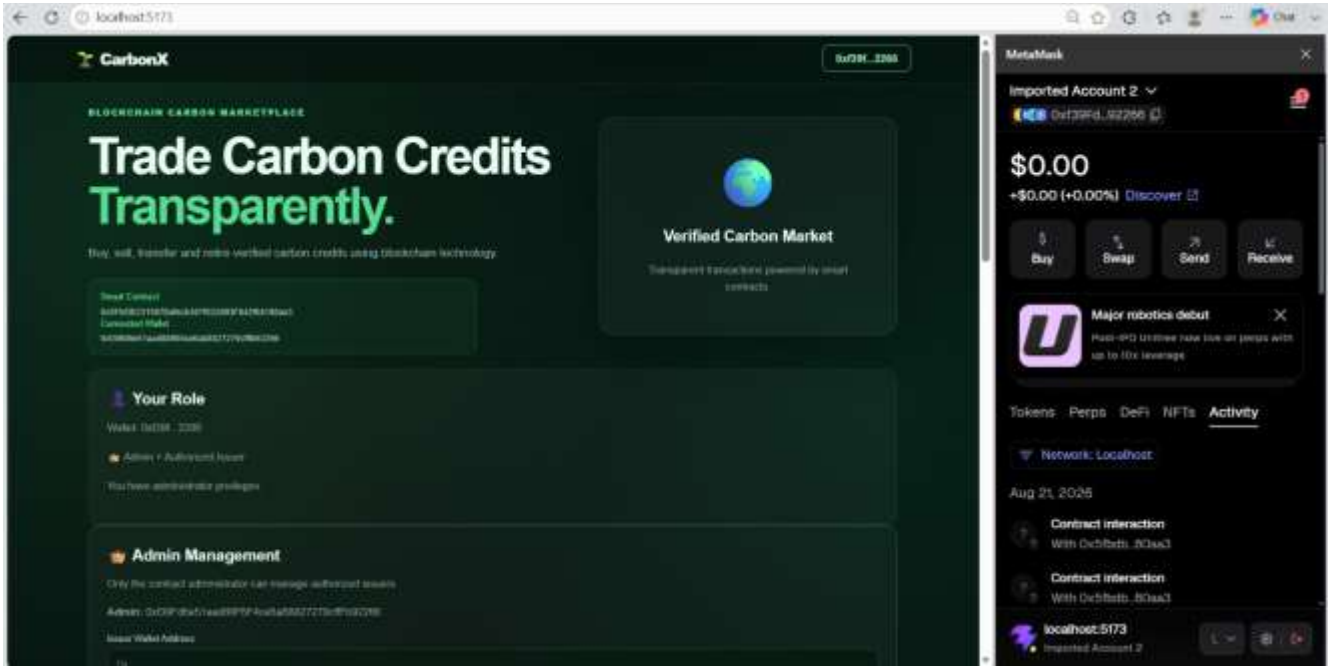
The CarbonX application was tested across different user roles and blockchain operations to verify that the main features work correctly.

- **Wallet Connection and Role-Based Interface**
  MetaMask wallet connection was tested, and available operations are displayed according to the connected account's role.
- **Issuer Registration and Credit Issuance**
  The administrator can register authorized issuers, and authorized issuers can create new carbon credits with the required project details.
- **Credit Information Retrieval**
  Users can enter a Credit ID and retrieve information such as project details, issuer, owner, tonnes of CO_2e, and current credit status.
- **Marketplace Listing and Status Checking**
  Credit owners can list eligible credits for sale and check whether a credit currently has an active marketplace listing.
- **Purchase and Ownership Update**
  A buyer can purchase an active listing through MetaMask. After a successful transaction, ownership is transferred to the buyer and the listing becomes inactive.
- **Listing Cancellation**
  The seller can cancel an active listing through a blockchain transaction. After cancellation, the listing becomes inactive.
- **Ownership Transfer**
  The current owner can transfer an eligible carbon credit to another wallet. The new owner is recorded on-chain.
- **Credit Retirement**
  The owner can permanently retire an eligible credit by providing a retirement reason. The credit status changes to RETIRED.

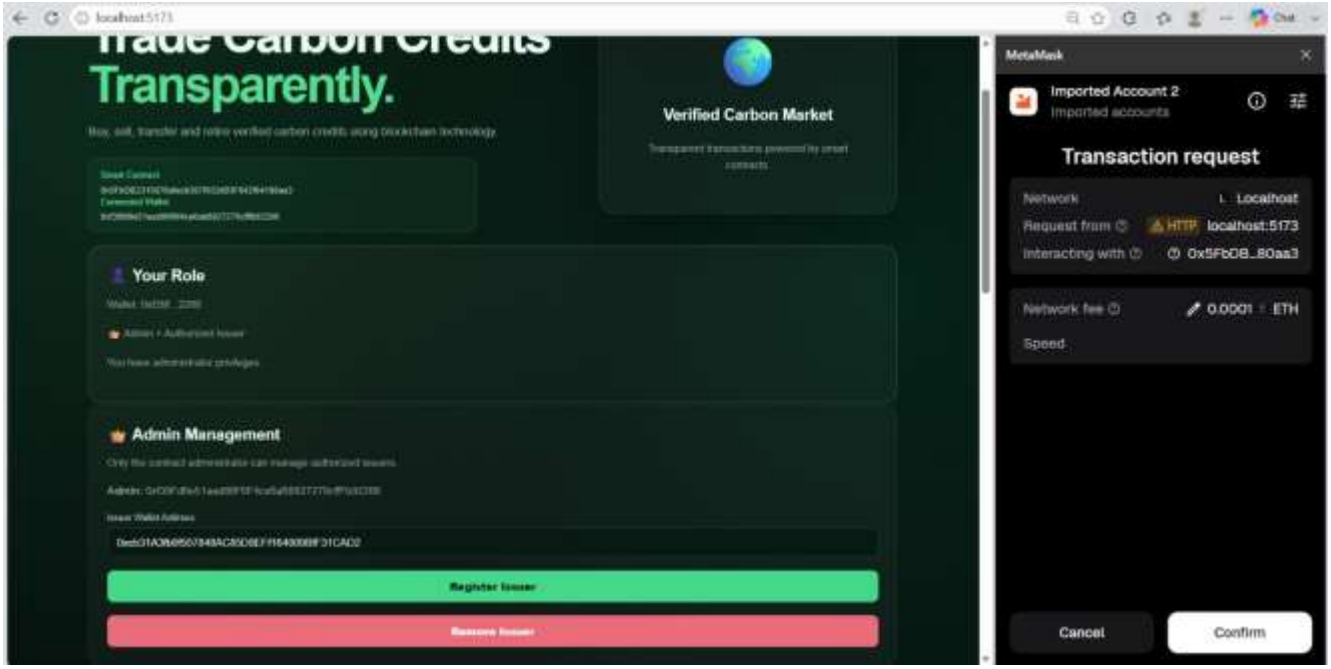
- **Prevention of Transfer and Listing After Retirement**
  The smart contract rejects attempts to transfer and list a credit after it has been retired.
- **MetaMask Transaction Confirmation**
  State-changing operations require MetaMask confirmation, providing a clear wallet-based approval process before blockchain transactions are executed.

17. Screenshot / Evidence Checklist

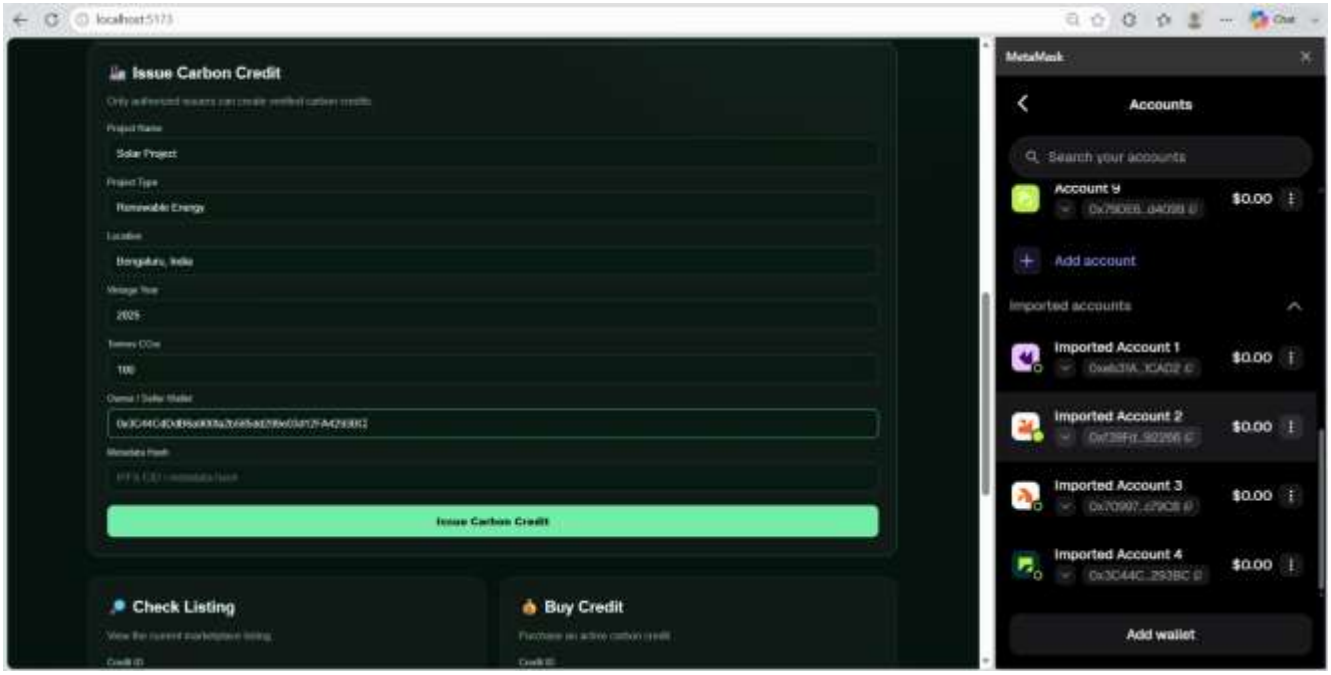
CarbonX Dashboard



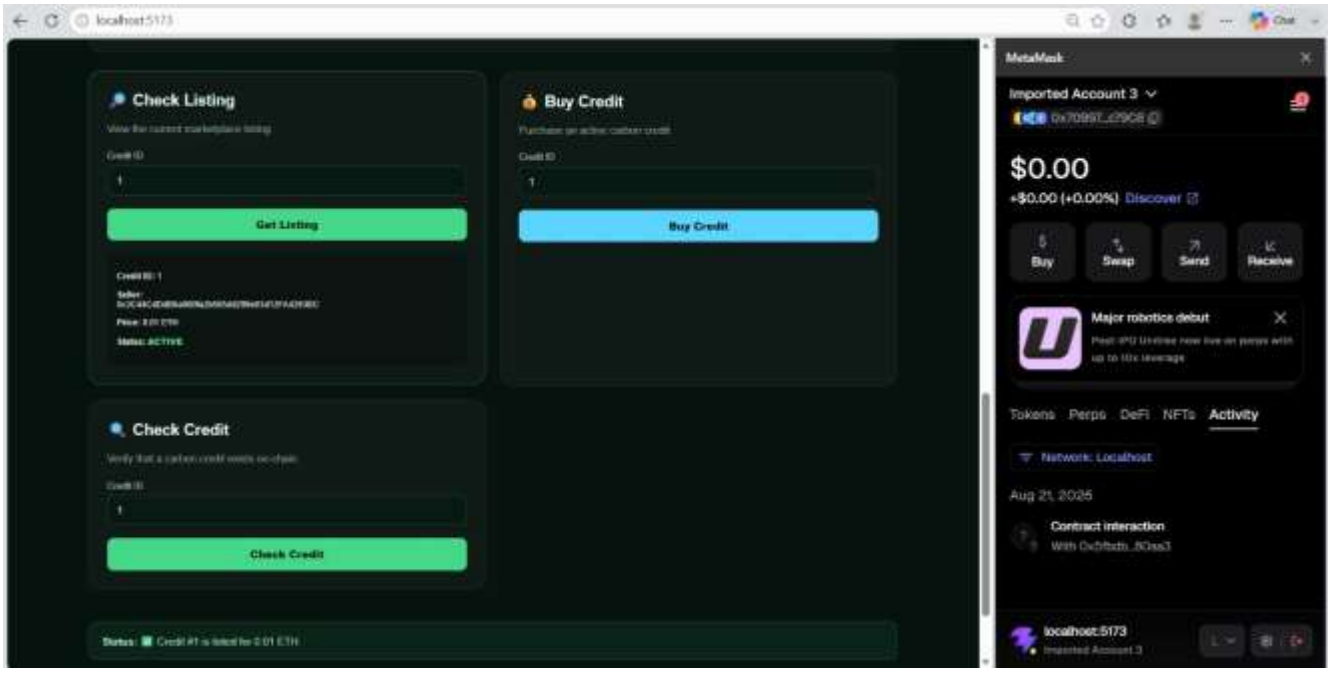
Admin and Authorized Issuer Role



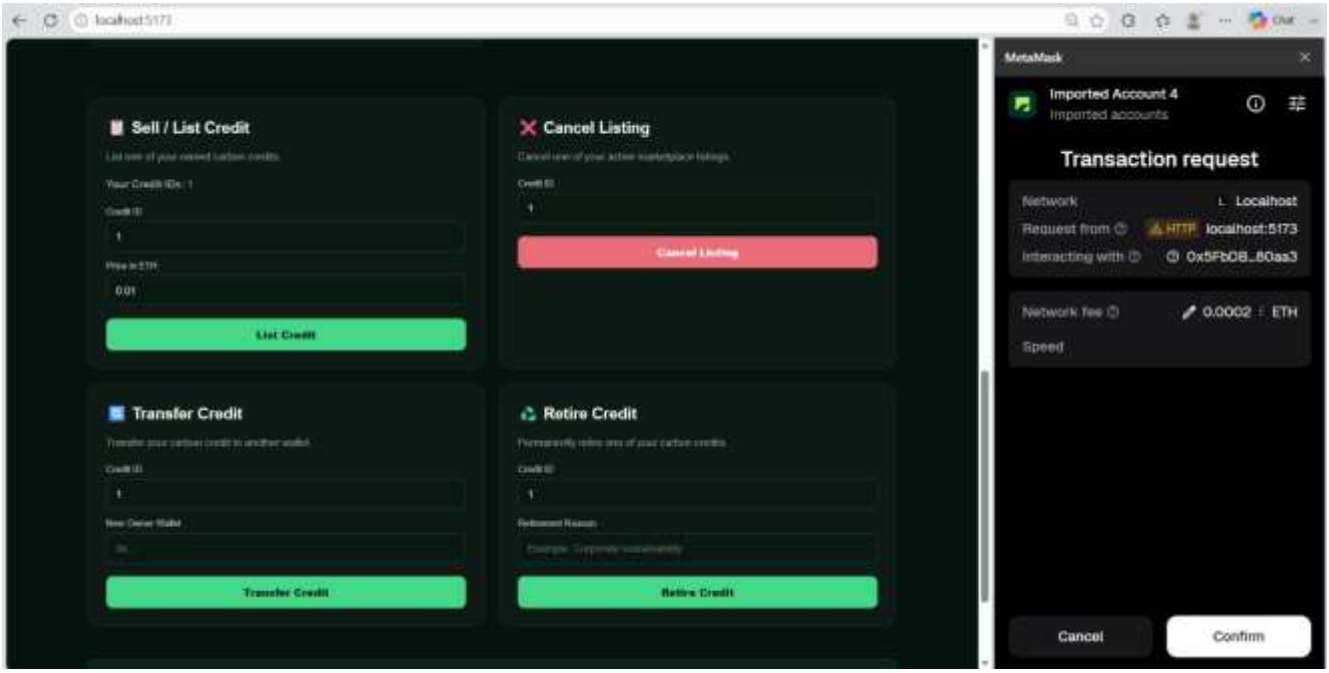
Issue Carbon Credit



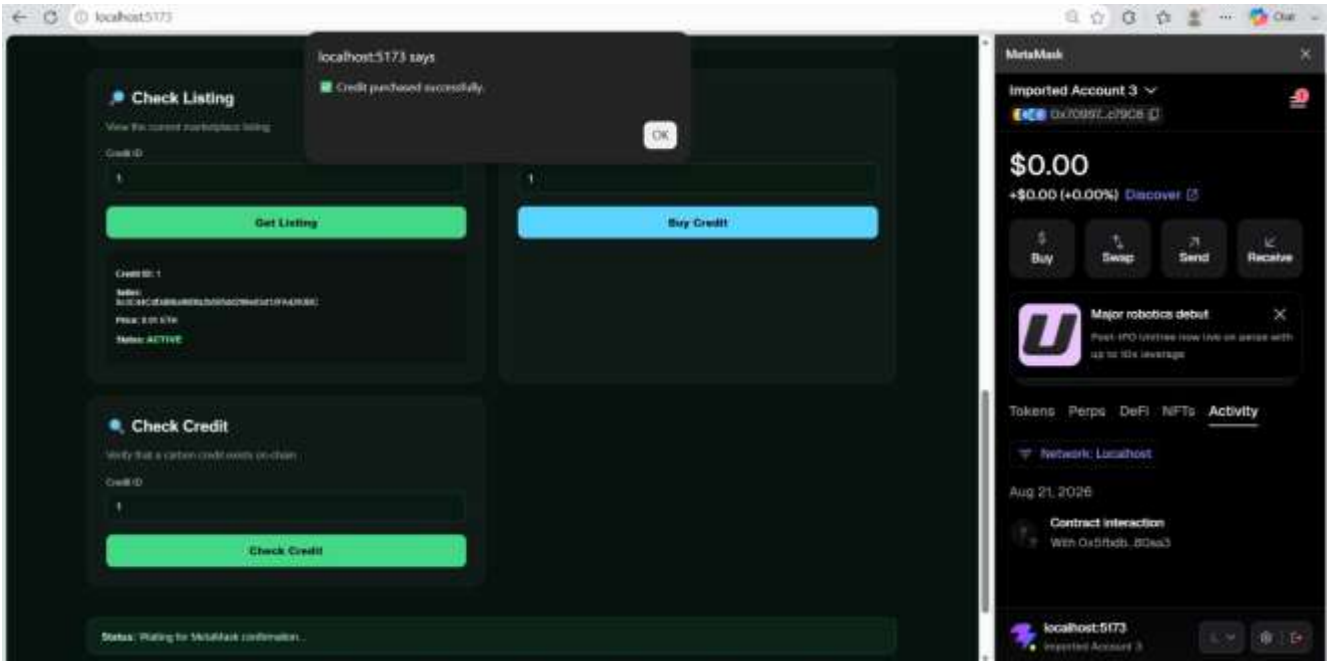
Check Carbon Credit



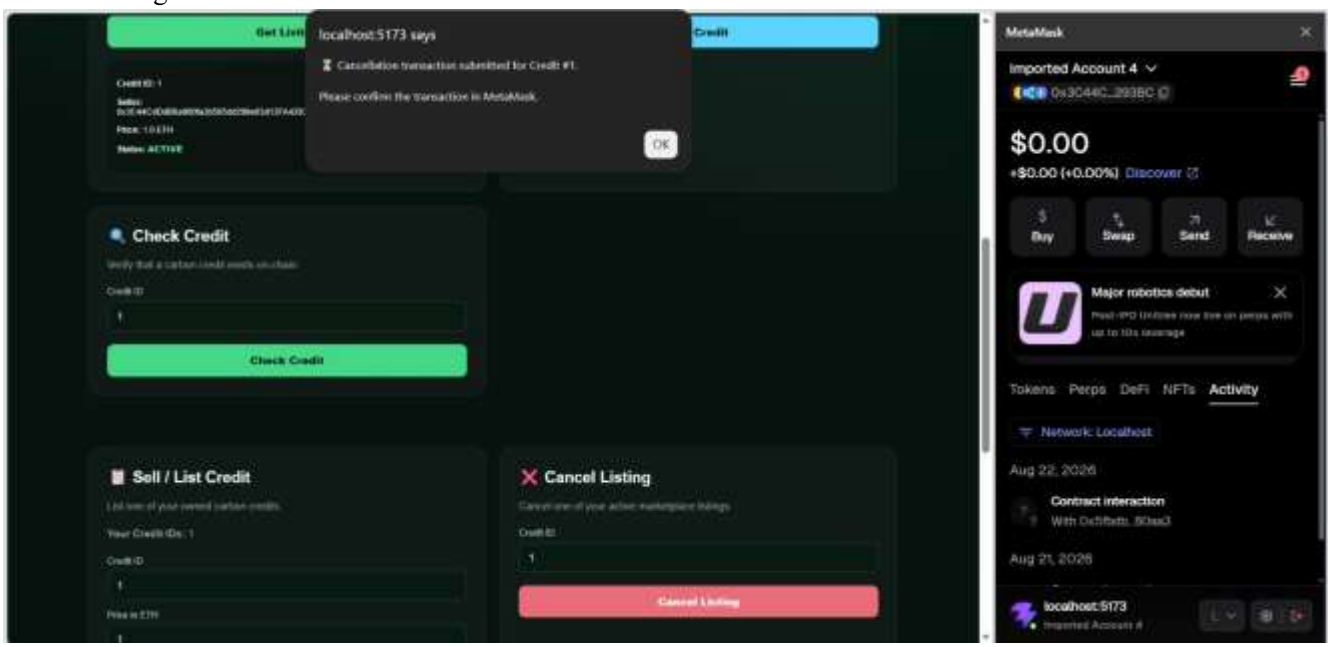
Sell / List Carbon Credit



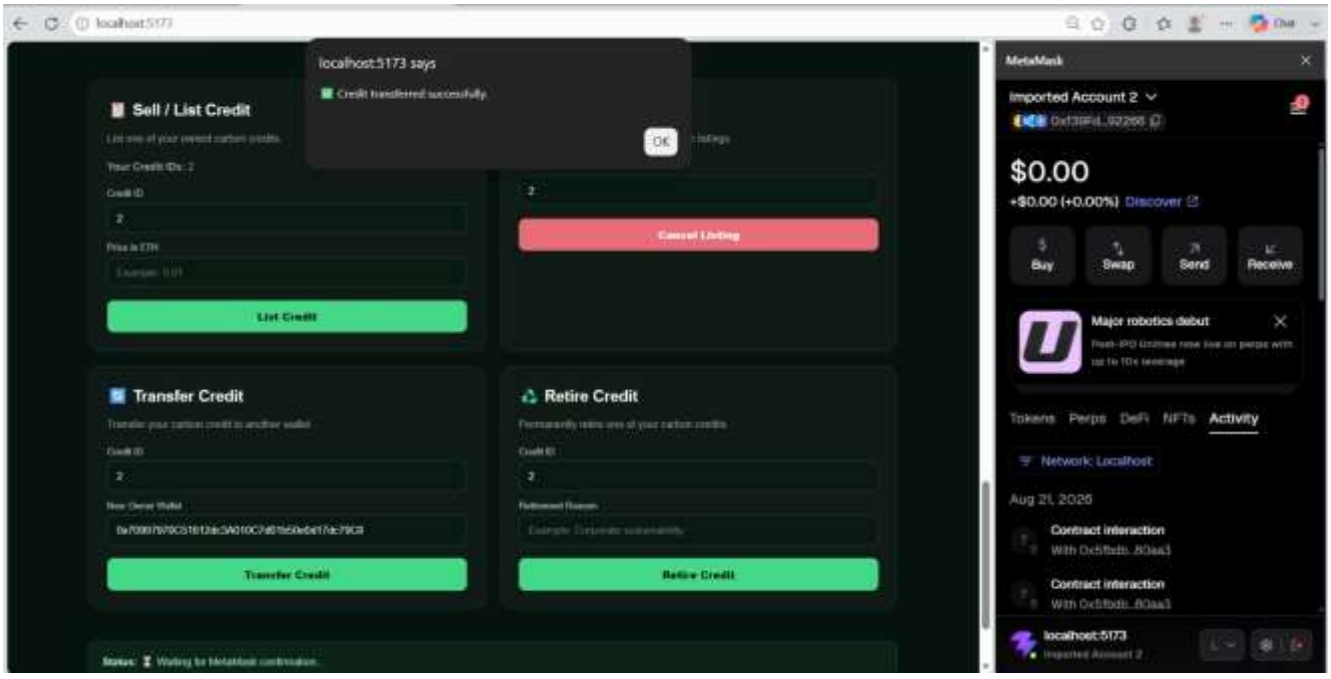
Successful Purchase



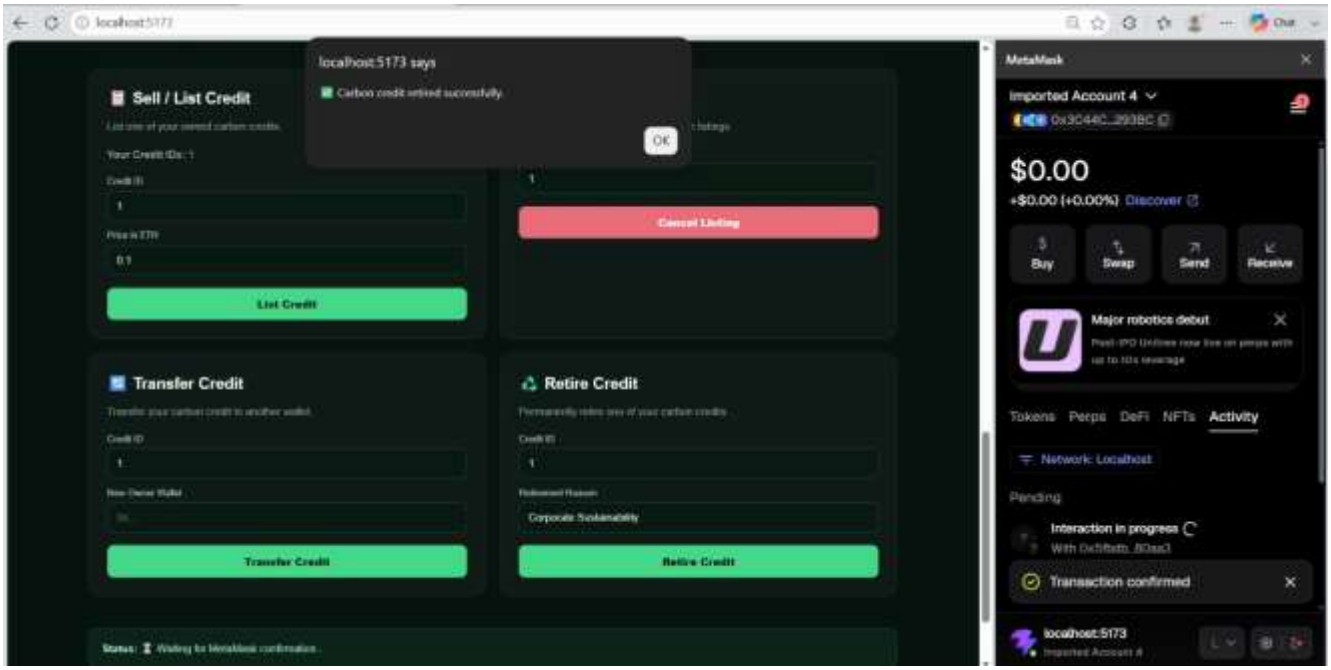
Cancel Listing



Successful Transfer



Successful Retirement



18. Advantages

- Transparent ownership and transaction records.
- Smart-contract-enforced access control.
- Traceable credit lifecycle.
- Automated marketplace state transitions.
- Permanent retirement state.
- Web3 wallet-based transaction approval.

19. Limitations

The project uses simulated carbon-credit information. Blockchain storage does not independently prove that a real-world project removed or avoided a specific amount of CO2. A production platform would require measurement, reporting, verification, independent auditing, approved methodologies, project validation, regulatory compliance, real-world monitoring, and integration with recognized registries.

20. Future Scope

- Project-document verification.
- Marketplace search and analytics.
- Price and transaction history.
- Digital retirement certificates.
- Blockchain explorer integration.
- Real-world carbon registry integration.
- Carbon impact dashboard.

21. Conclusion

CarbonX demonstrates an educational prototype for blockchain-based carbon-credit management covering issuance, verification, marketplace trading, ownership transfer, cancellation, retirement, and retired-credit protection.

22. Author

Shubhashree Baburaya Nayak
GitHub: shubha229/Blockchain-Based-Carbon-Credit-Trading